EE2016 Microprocessors Theory and Lab, Aug - Nov 2025

Tutorial 5 (Classes on 1st, 3rd, 8th Sep - 1st batch (29th Aug, 2nd, 9th Sep - 2nd batch))

Dr. R. Manivasakan, EE Dept, IIT Madras.

Portions: Keyboard debouncing problem, polling implementation in KB debouncing problem, interrupt implementation in KB debouncing problem, and polling (KIN bit) and interrupt (KIRQ) implementation in muP - KB PIC problem, CU design, microprogramming and hardwired control unit, hardwired CU design examples: #1 Booths multiplier, #2 GCD processor, hardware design procedure - abstraction of CU as an FSM - Moore's (rather than Mealy's) machine, consolidation of states involved, table of current state versus next states and value of output signals (Boolean variables), translation of above states into excitation table (TT) and subsequent digital design.

1 Solve

1. Consider the design of a hardwired control unit (CU) based on the classical method, for an FPU which ONLY adds or subtracts two floating point numbers in standards representation. Assume the Moore's machine for FSM associated with the Add/Sub ONLY FPU. Given the following information,

   (a) Show the CU with input and output control signals for the above datapath hardware.

   (b) Derive the CU for the above Add/Sub only FPU processor by the standard procedure (taught in the class, steps given below) after assuming Moore type FSM.

      i. Identify the right FSM modelling the algorithm (or pseudocode)
      ii. Identify the states and write the corresponding state table
      iii. Identify the excitation table (Truth table)
      iv. (Optional) Using Karnaugh's map derive the expression for each output
      v. (Optional) Using the result in 1(d)(iv) draw the corresponding circuit

   Following are **the information** on steps of *floating-point addition/ subtraction*: the addition of two floating-point numbers, $A = M_A \times 2^{E_A}$ and $B = M_B \times 2^{E_B}$, is a multi-step process that ensures alignment before computation and normalization afterward.

   (a) *Step 1: Check for Zeros and Special Cases*

      i. If either operand is zero, the result is the other operand.
      ii. Check for other special values like Infinity or NaN (Not a Number).

   (b) *Step 2: Compare Exponents and Align Mantissas (Normalization)*

      i. *Find the Difference:* Calculate the difference between the two exponents: $\Delta_E = |E_A - E_B|$.
      ii. *Identify the Smaller Exponent*: Determine which number has the smaller exponent.
      iii. *Shift the Smaller Mantissa*: The mantissa corresponding to the smaller exponent must be right-shifted by Δ bits. This effectively aligns the binary point of the two mantissas.
         A. Example: If $E_A = 5$ and $E_B = 3$, the mantissa M_B is right-shifted by $5 - 3 = 2$ bits.
      iv. Set the Result Exponent: The exponent of the result (E_R) is set to the larger of the two original exponents ($E_R = max\,(E_A, E_B)$).

   (c) *Step 3: Add/Subtract the Mantissas*

      i. Perform the actual arithmetic operation on the aligned mantissas.

A. If the original floating-point numbers had the same sign, add the mantissas: $M_R = M_{A^*} + M_{B^*}$ where M_{A^*} and M_{B^*} are aligned respective mantissas.

B. If they had different signs, subtract the mantissas: $M_R = |M_{A^*} - M_{B^*}|$.

ii. The sign of the result (S_R) is determined by the number with the larger magnitude.

(d) *Step 4: Normalize the Result*

The resulting mantissa (M_R) must be normalized so that its most significant '1' bit is immediately to the left of the binary point (assuming a standard representation).

i. If $M_R > 2$: The result is too large. Right-shift M_R by 1 bit and increment the exponent E_R by 1.

ii. If $M_R < 1$: The result is too small. Left-shift M_R until the most significant '1' is in the correct position. For every bit shifted left, decrement the exponent E_R by 1.

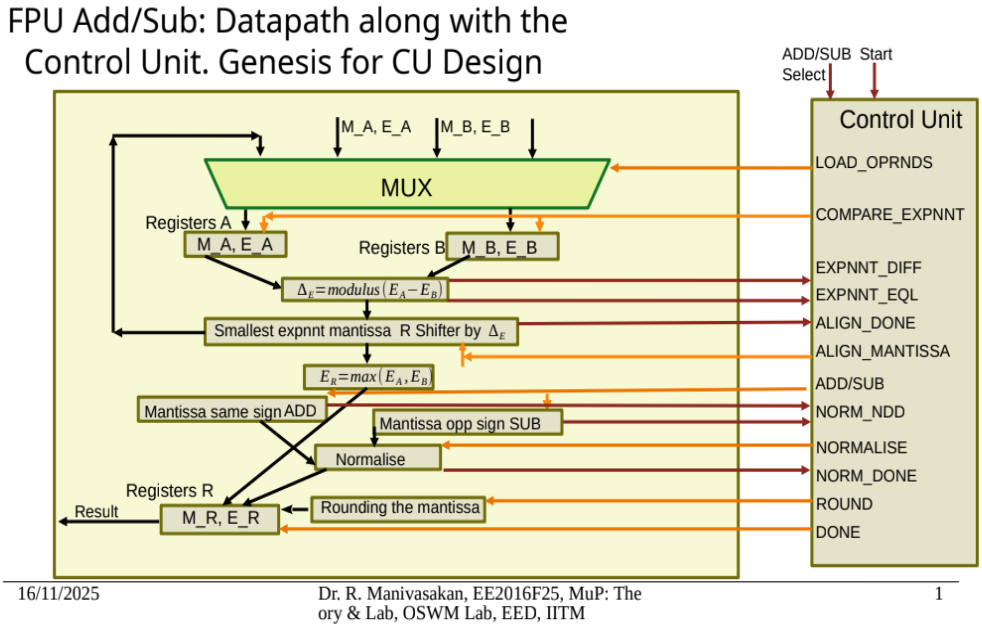
iii. Check for Underflow/Overflow: If the exponent E_R becomes too small (Underflow) or too large (Overflow), the appropriate exception must be handled.

(e) *Step 5: Round the Mantissa*

The normalized mantissa must be rounded to fit the precision allowed by the floating-point standard (e.g., 23 bits for IEEE 754 single precision). Rounding rules (e.g., Round to Nearest Even) are applied using the discarded bits (guard, round, and sticky bits).

Solutions:

(a) Here it goes (This picture allows you to understand the datapath & its interaction with CU - very similar to the one I drew for the design of CU for Booth's algorithm):



(b) Answer.

i. FSM diagram is simple. Look at the state table, states are nodes and draw the jump from current state to next state. The branch is marked by the input. For instance S0 —-START=1—> S1. Like this every entry in the state table is to be covered.

ii. The states of the above FSM are

A. 000 S0: IDLE (Waiting for START signal)

B. 001 S1: LOAD (Load operands into FPU registers)

C. 010 S2: COMPARE (Compare exponents to determine alignment)

D. 011 S3: ALIGN (Align mantissas by shifting smaller exponent)

E. 100 S4: OPERATE (Perform addition or subtraction on mantissas)

F. 101 S5: NORMALIZE (Normalize result (shift and adjust exponent)

G. 110 S6: ROUND (Round the result according to rounding mode)

H. 111 S7: STORE (Store result and set DONE flag)

The corresponding state table is

S. No	Current State	Condition / Input	Next State	Control Outputs
	S0 (000)	START = 0	S0 (000)	All outputs = 0
	S0 (000)	START = 1	S1 (001)	BUSY = 1
	S1 (001)	Always	S2 (010)	LOAD_OP = 1, BUSY = 1
	S2 (010)	Always	S3 (011)	COMP_EXP = 1, BUSY = 1
	S3 (011)	EXP_EQUAL = 1	S4 (100)	BUSY = 1
	S3 (011)	EXP_EQUAL = 0, ALIGN_DONE = 0	S3 (011)	ALIGN_MANT = 1, BUSY = 1
	S3 (011)	EXP_EQUAL = 0, ALIGN_DONE = 1	S4 (100)	BUSY = 1
	S4 (100)	ADD_DONE = 0	S4 (100)	ADD_SUB = 1, BUSY = 1
	S4 (100)	ADD_DONE = 1	S5 (101)	BUSY = 1
	S5 (101)	NORM_NEEDED = 0	S6 (110)	BUSY = 1
	S5 (101)	NORM_NEEDED = 1, NORM_DONE = 0	S5 (101)	NORMALIZE = 1, BUSY = 1
	S5 (101)	NORM_NEEDED = 1, NORM_DONE = 1	S6 (110)	BUSY = 1
	S6 (110)	Always	S7 (111)	ROUND = 1, BUSY = 1
	S7 (111)	Always	S0 (000)	STORE_RES = 1, DONE = 1, BUSY = 0